

1 Write a Python program to create a class with constructor and object creation.

AIM

To write a Python program that defines a class with a constructor () and demonstrates object creation.

ALGORITHM

1. Start the program.
2. Define a class (e.g.,).
3. Create a constructor () that initializes attributes like name and age.
4. Define a method to display student details.
5. In the main program, create objects of the class.
6. Call methods to display information.
7. End program.

CODE

```
# Class with Constructor and Object Creation
```

```
class Student:
```

```
    # Constructor to initialize attributes
```

```
    def __init__(self, name, age, course):
```

```
        self.name = name
```

```
        self.age = age
```

```
        self.course = course
```

```
    # Method to display student details
```

```
    def display(self):
```

```
        print(f"Name: {self.name}, Age: {self.age}, Course: {self.course}")
```

Object Creation

```
student1 = Student("Yogin", 20, "Computer Science")
```

```
student2 = Student("Amit", 22, "Mechanical Engineering")
```

Display details

```
print("Student Details:")
```

```
student1.display()
```

```
student2.display()
```

OUTPUT

Student Details:

Name: Yogin, Age: 20, Course: Computer Science

Name: Amit, Age: 22, Course: Mechanical Engineering

CONCLUSION

This program demonstrates:

- Class creation ().
- Constructor () to initialize object attributes.
- Object creation (,).
- Method calling to display details.

📖 This shows how Object-Oriented Programming (OOP) in Python allows us to model real-world entities with attributes and behaviors.

2 Write a class to calculate the area of a rectangle using methods.

AIM

To write a Python program that defines a class with methods to calculate the area of a rectangle.

ALGORITHM

1. Start the program.
2. Define a class .
3. Create a constructor () to initialize length and width.
4. Define a method that returns the area ().
5. In the main program, create objects of the class.
6. Call the method to calculate and display the area.
7. End program.

CODE

```
# Class to calculate area of a rectangle
```

```
class Rectangle:
```

```
    # Constructor to initialize length and width
```

```
    def __init__(self, length, width):
```

```
        self.length = length
```

```
        self.width = width
```

```
    # Method to calculate area
```

```
    def calculate_area(self):
```

```
        return self.length * self.width
```

```
# Object Creation
```

```
rect1 = Rectangle(10, 5)
```

```
rect2 = Rectangle(7, 3)
```

Display results

```
print("Area of Rectangle 1:", rect1.calculate_area())
```

```
print("Area of Rectangle 2:", rect2.calculate_area())
```

OUTPUT

Area of Rectangle 1: 50

Area of Rectangle 2: 21

CONCLUSION

This program demonstrates:

- Class creation ().
- Constructor () to initialize attributes.
- Method () to perform calculation.
- Object creation and method calling to compute results.

📖 This shows how Object-Oriented Programming (OOP) can model real-world entities like rectangles and perform operations using methods.